



AMERICAN
UNIVERSITY
OF BEIRUT

Fall 2025

EECE 490: Introduction to Machine Learning

C5 - Unsupervised Machine Learning Algorithms

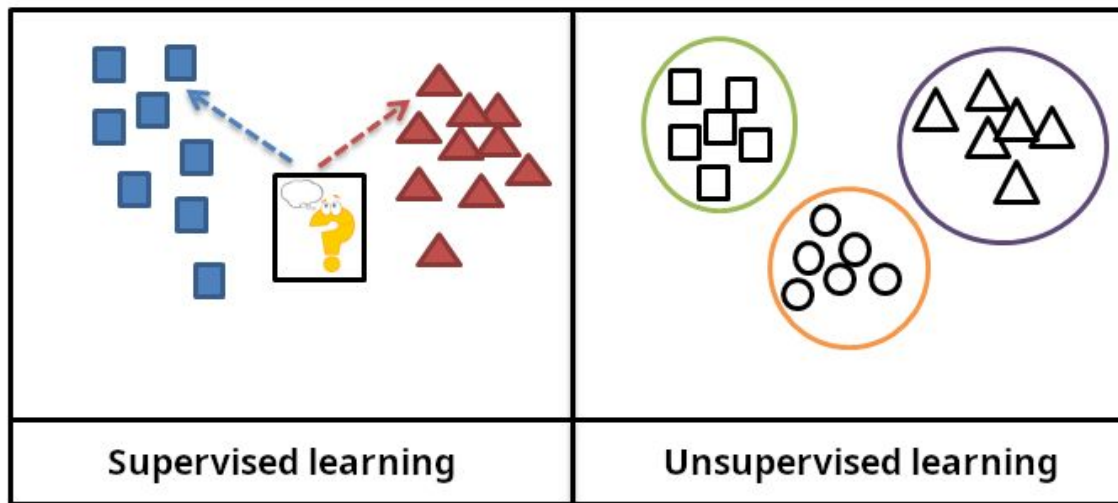
What is Unsupervised Machine Learning (UML)?

Previously, in supervised machine learning, we were dealing with **labeled** data. This means that we were **training the model to predict a specified output feature**.

Alternatively, unsupervised machine learning uses **unlabeled** data. The purpose is not to perform a prediction, but to understand, analyze, and cluster the data.

What is Unsupervised Machine Learning (UML)?

Unsupervised learning works with data that has no predefined labels or categories. The primary goal is to explore the underlying structure of the data, allowing the model to learn from the data itself.



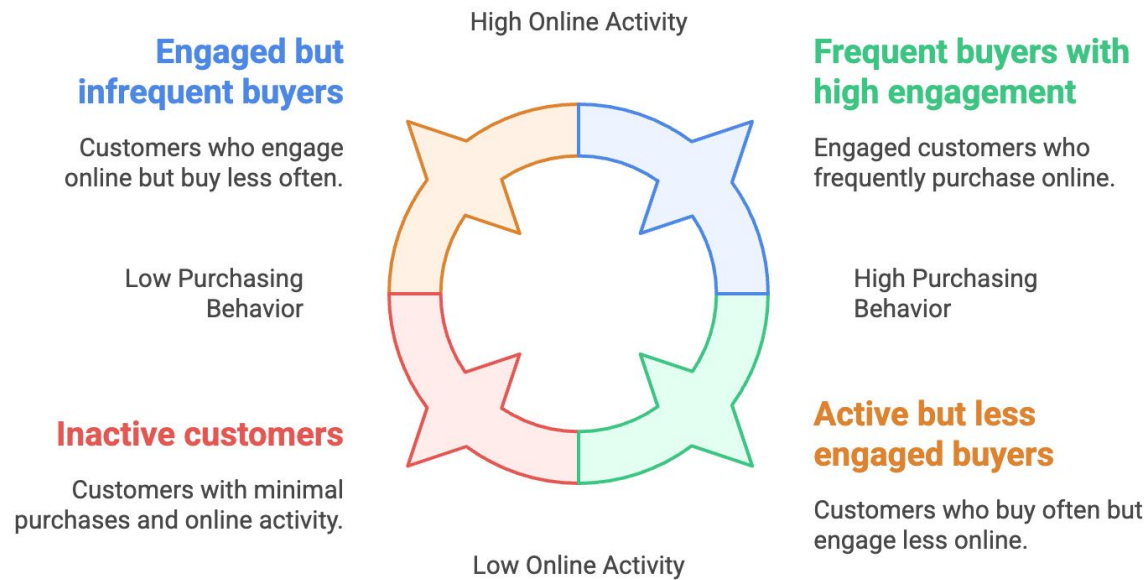
Applications of Unsupervised Machine Learning

Customer Segmentation in Marketing

Identify customer groups based on purchasing behavior, demographics, and online activity.

Example:

- Amazon
- Ali Express



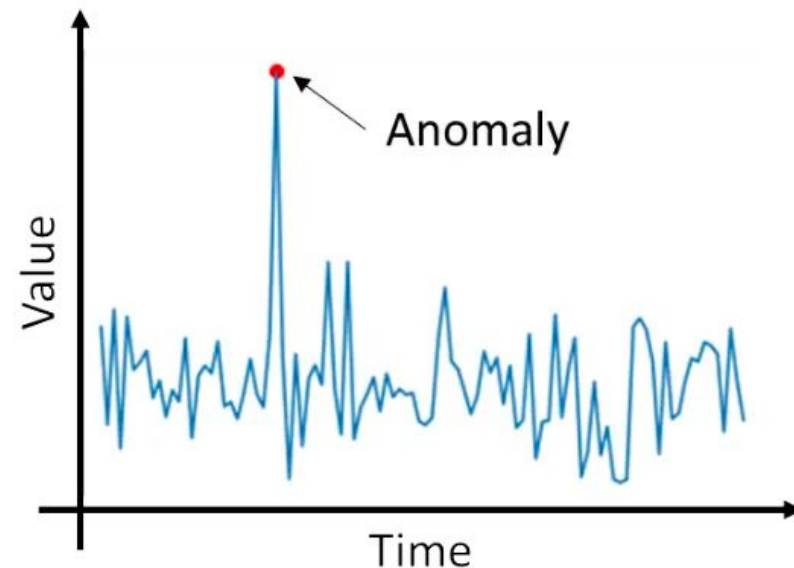
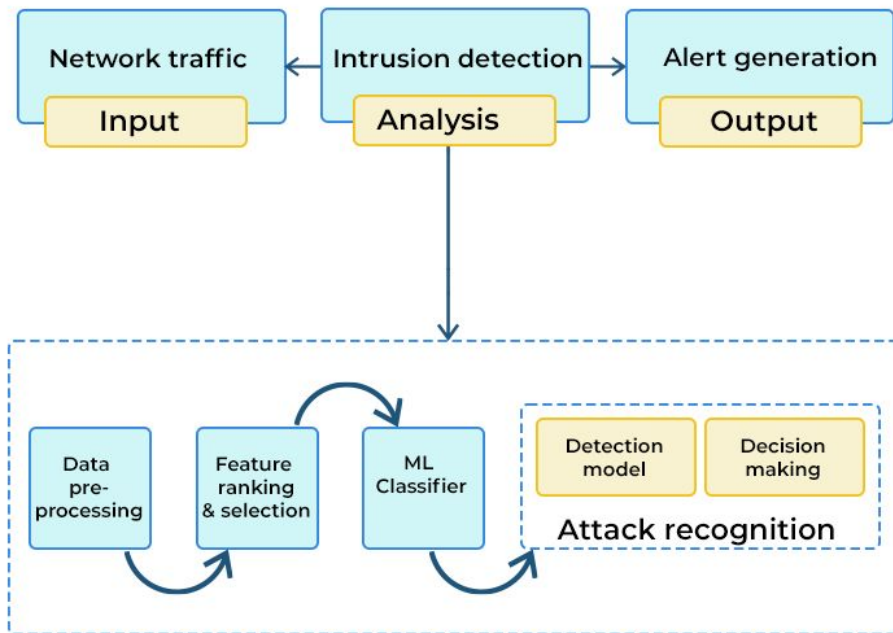
Customer Segmentation in Marketing

The benefits of using UML to segment customers:

- Tailored marketing campaigns
- Improved customer service and targeted promotions
- Enhanced customer experience and retention

Anomaly Detection in Cybersecurity

SYSTEM ARCHITECTURE OF NETWORK ANOMALY DETECTION SYSTEM



Anomaly Detection in Cybersecurity

The benefits of using UML in cybersecurity:

- Early detection of cyber-attacks and fraud
- Real-time monitoring of network activity
- Reduced false-positive rates by learning normal behavior patterns

Big Data Analysis

Reduce high-dimensional data to lower dimensions for visualization and analysis.



Types of Unsupervised Machine Learning Algorithms

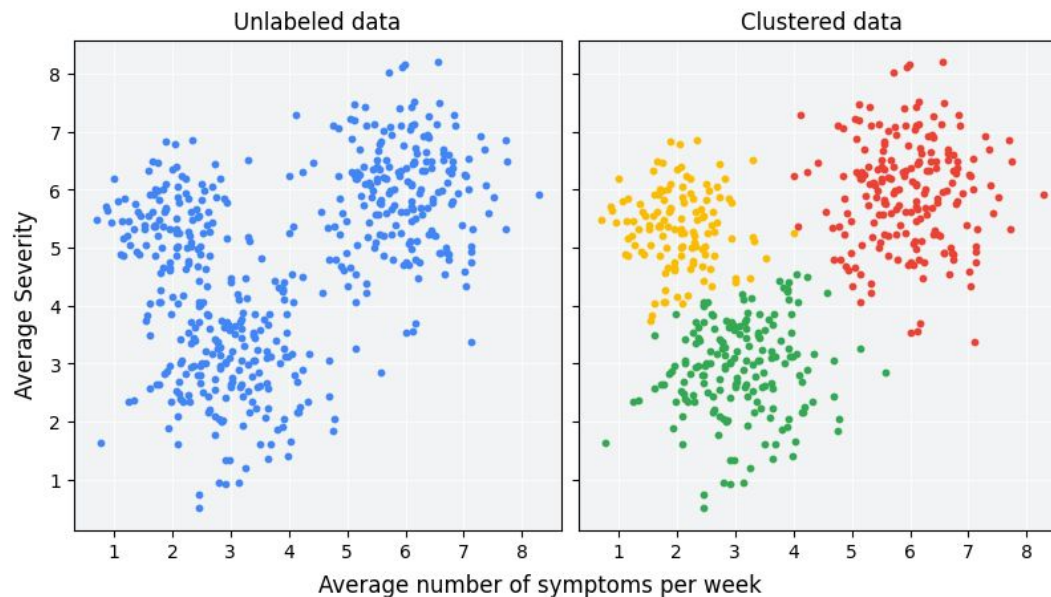


Clustering Algorithms

Clustering

Clustering is a method of finding **similarity groups that are called clusters**.

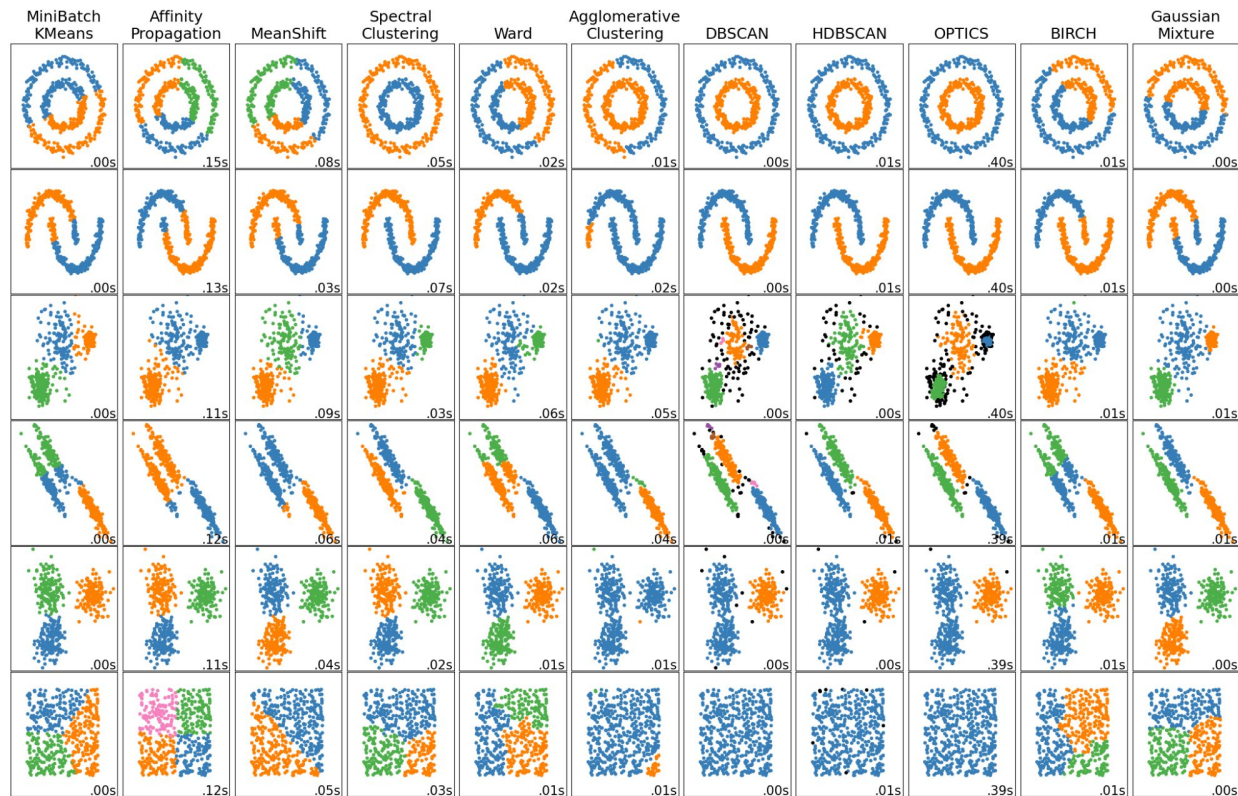
It groups data instances that are similar to (near) each other in one cluster and data instances that are very different (far away) from each other into different clusters.



Clustering

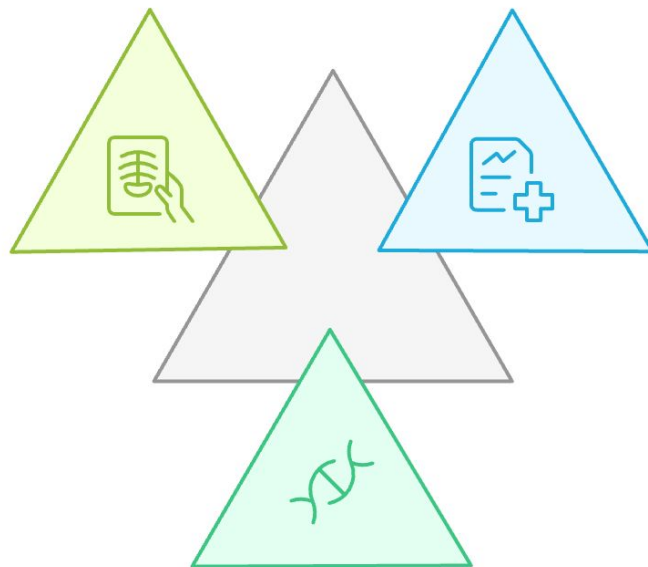


The clusters are not always obvious, and different clustering algorithms will yield different outcomes



Applications of clustering algorithms

Clustering Applications in Healthcare



Patient Records

Organizing patient data for better insights



Gene Expression Data

Analyzing genetic information for patterns

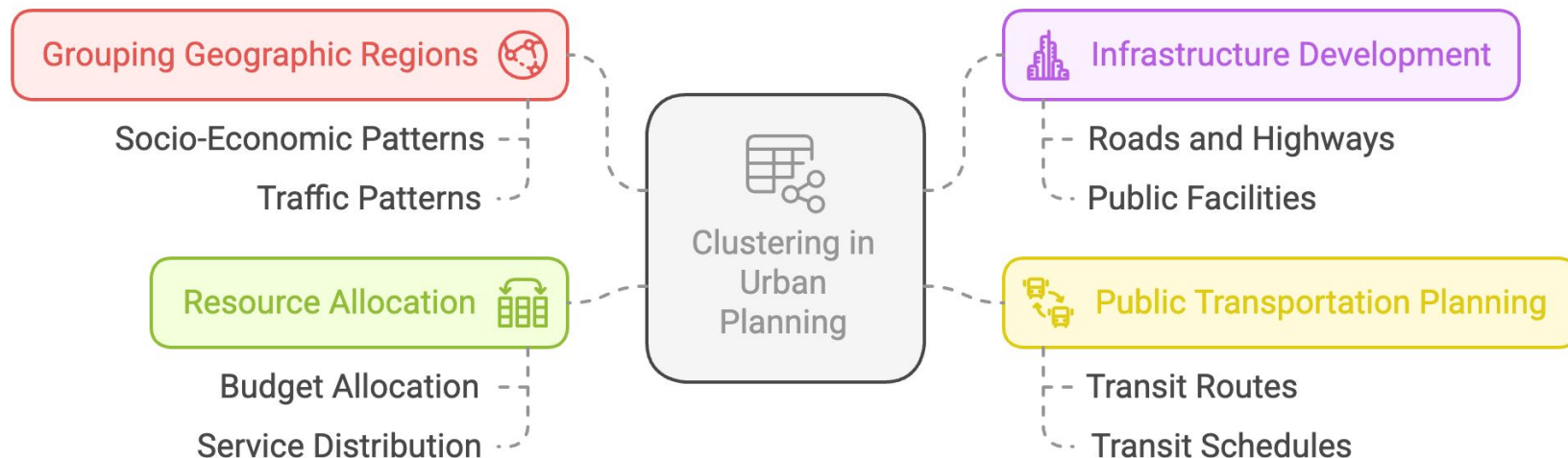


Medical Images

Enhancing image analysis for diagnosis

Applications of clustering algorithms

Clustering in Urban Planning and Smart Cities

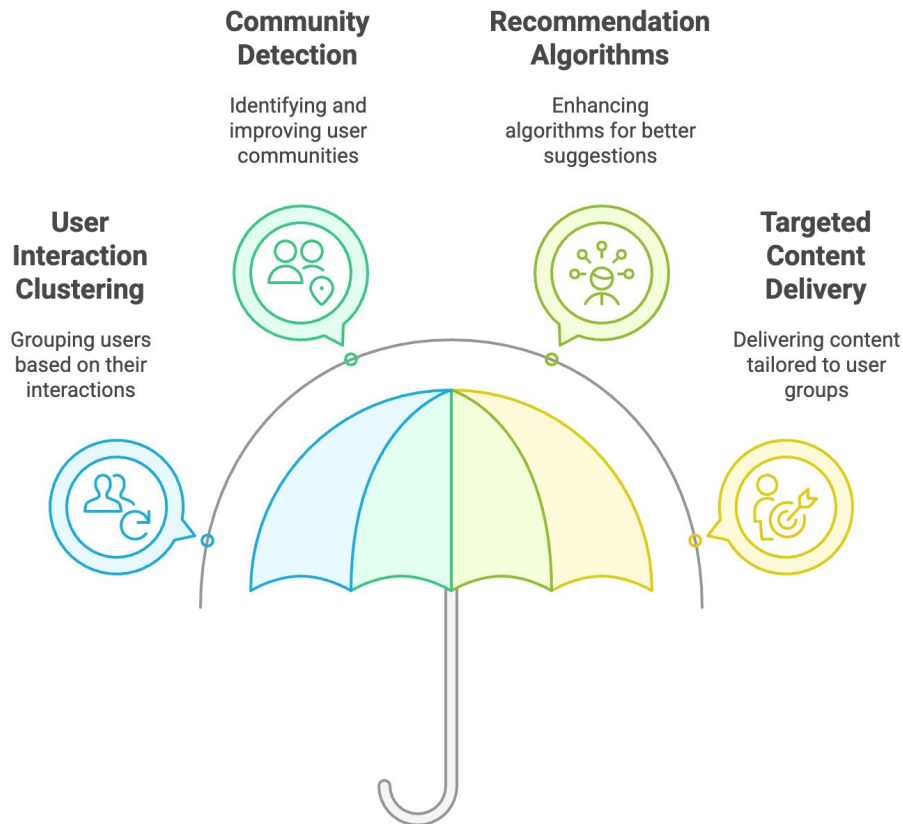


Applications of clustering algorithms



AMERICAN
UNIVERSITY
OF BEIRUT

Enhancing Social Networks with Clustering



Companies that use clustering

Amazon:

- Usage: Clusters customers based on purchasing behavior and browsing history.
- Benefits: Enables personalized product recommendations, tailored marketing campaigns, and improved inventory management.



Companies that use clustering

Spotify:

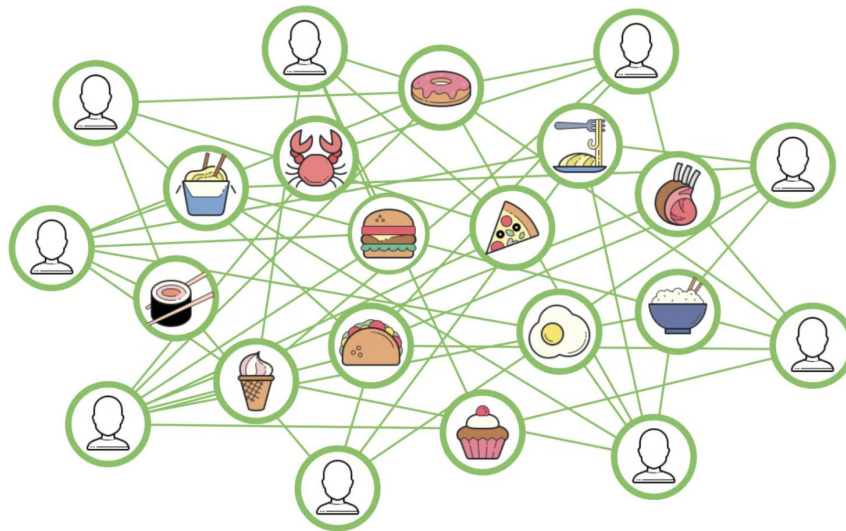
- Usage: Uses clustering to group songs into similar genres or moods and to segment listeners based on their musical preferences.
- Benefits: Drives personalized playlist creation and music recommendation, improving user engagement.



Companies that use clustering

Uber:

- Usage: Clusters geographical regions based on ride demand and travel patterns.
- Benefits: Optimizes driver deployment, reduces wait times for customers, and increases operational efficiency.



Clustering algorithms

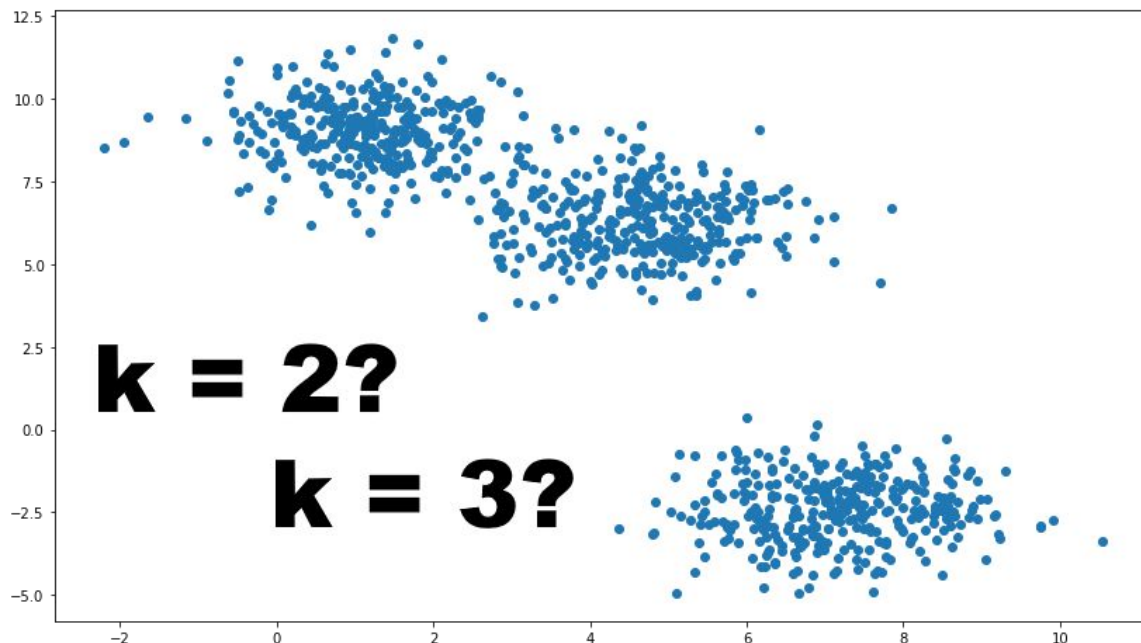
In this section, we will go through 4 clustering algorithms:

1. K-means clustering
2. DBSCAN algorithm
3. Hierarchical clustering
4. Spectral clustering



K-means clustering

K-means clustering create k clusters from the given data. K is a hyperparameter that we set and tune until we get the most informative results.

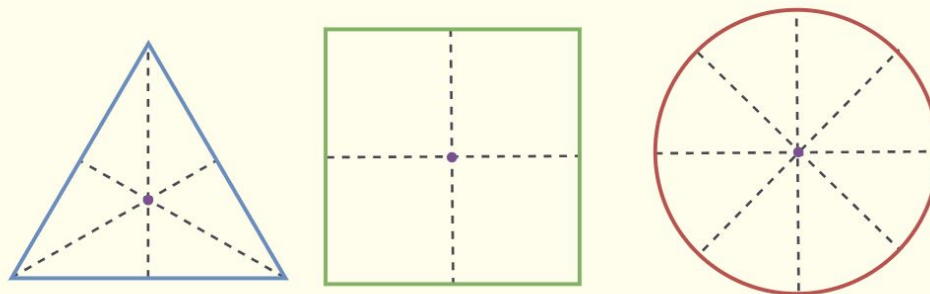


K-means clustering

K-means clustering uses centroids to tune and create the final clusters. The idea is that the centroids of each cluster should be equal to the average of all the data points inside the cluster.

Recap: The centroid is the centre point of the object.

Centroid in Geometry

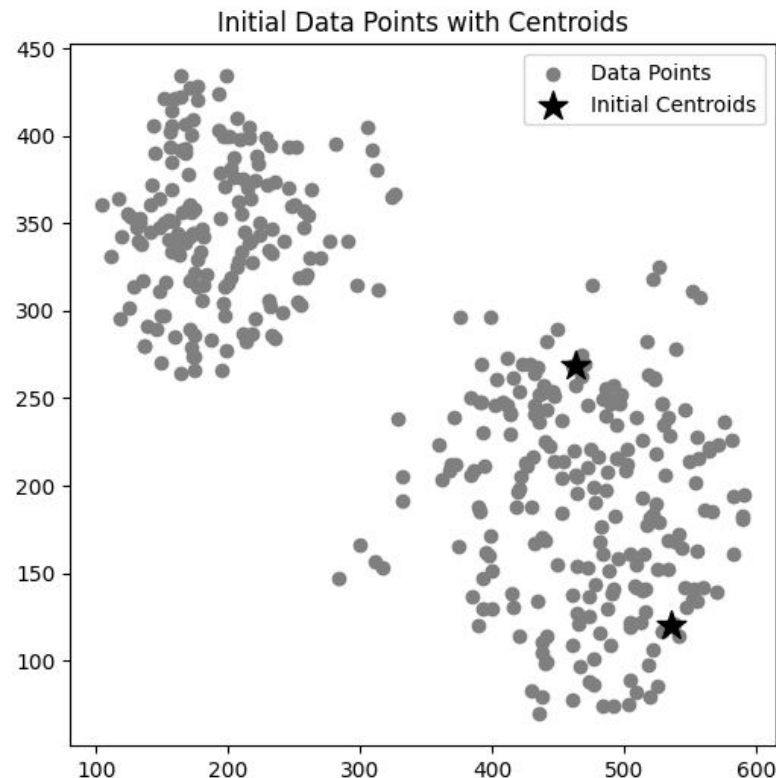


K-means clustering

So, how does the k-means clustering algorithm choose the clusters?

First, we initialize k centroids of random positions in the data.

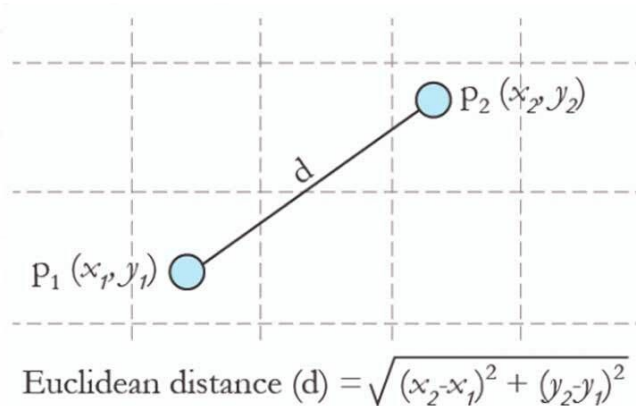
In this example, we will set $k=2$



K-means clustering

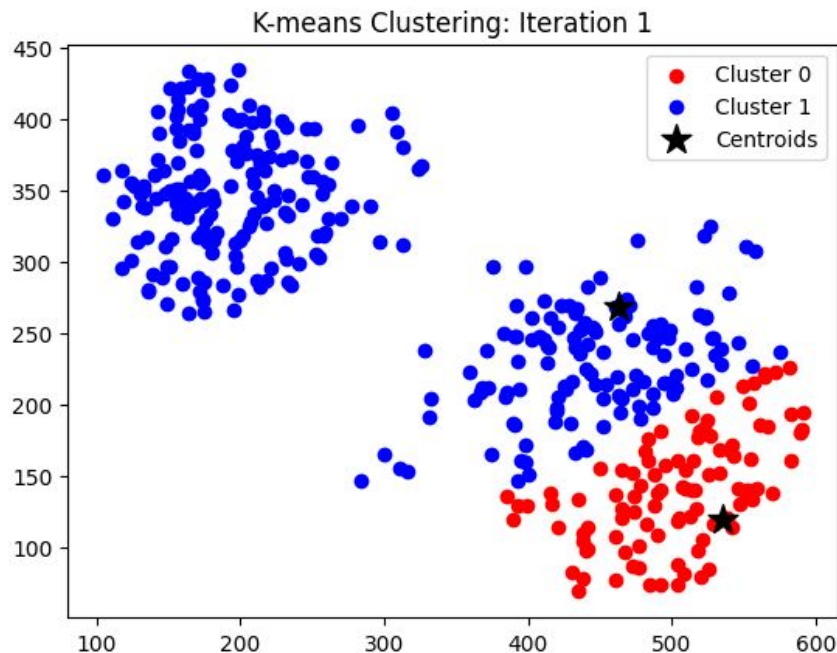
After initializing the centroids, we need to identify which centroid each data point belongs to.

We do this through euclidean distance. If $\text{distance}(\text{data_point}, \text{centroid_1}) < \text{distance}(\text{data_point}, \text{centroid_2})$. Then, the data point belongs to cluster 1, which has a centroid centroid_1 .



K-means clustering

Thus, we create the initial clusters by computing the euclidean distance between each data point and the randomly initialized centroids.

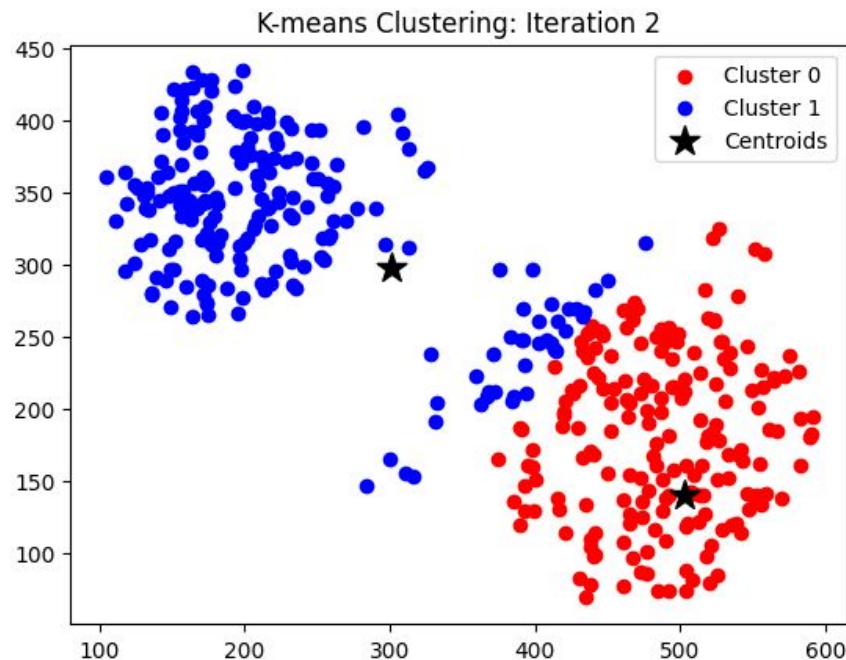


K-means clustering

After we have the two initial clusters, we compute the average of each of those clusters.

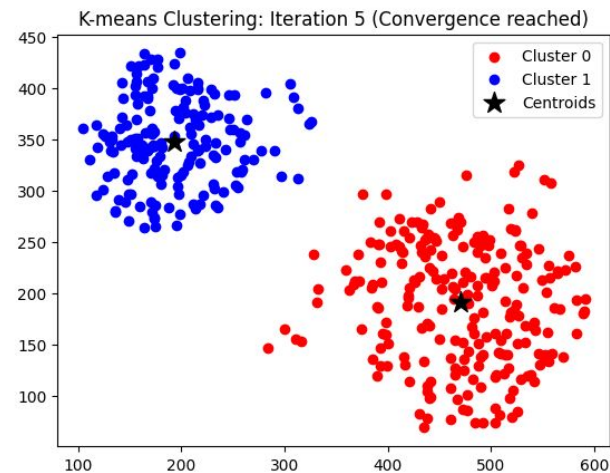
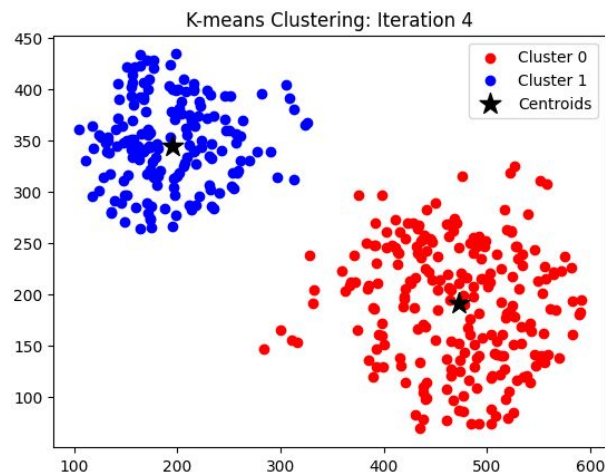
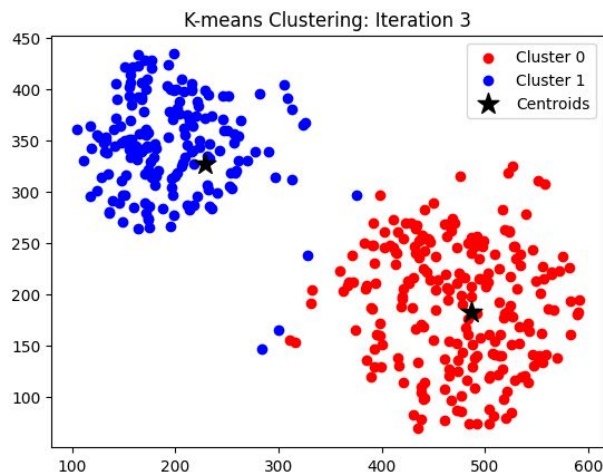
This average will be the new centroid.

Using euclidean distance, we will then generate the two new clusters.



K-means clustering

We keep on repeating this process until the average of the clusters does not change.



K-means clustering

To recap, these are the training steps for k-means clustering:

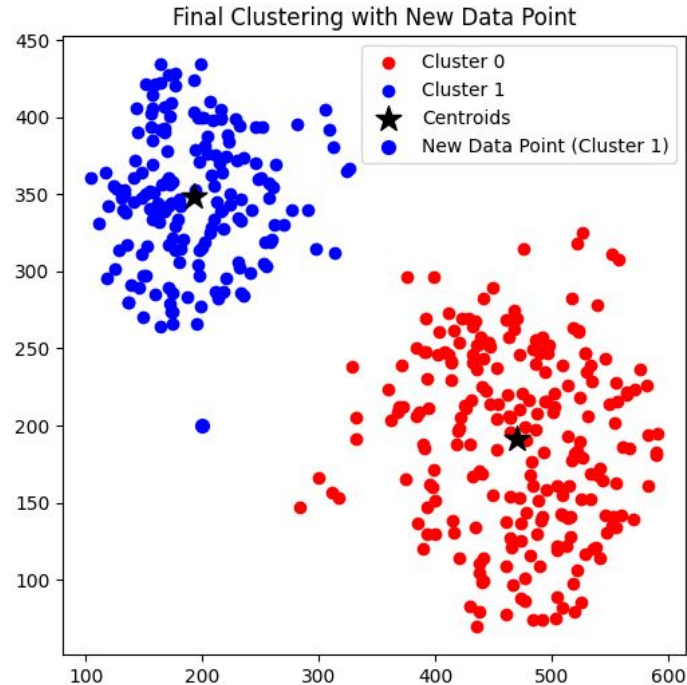
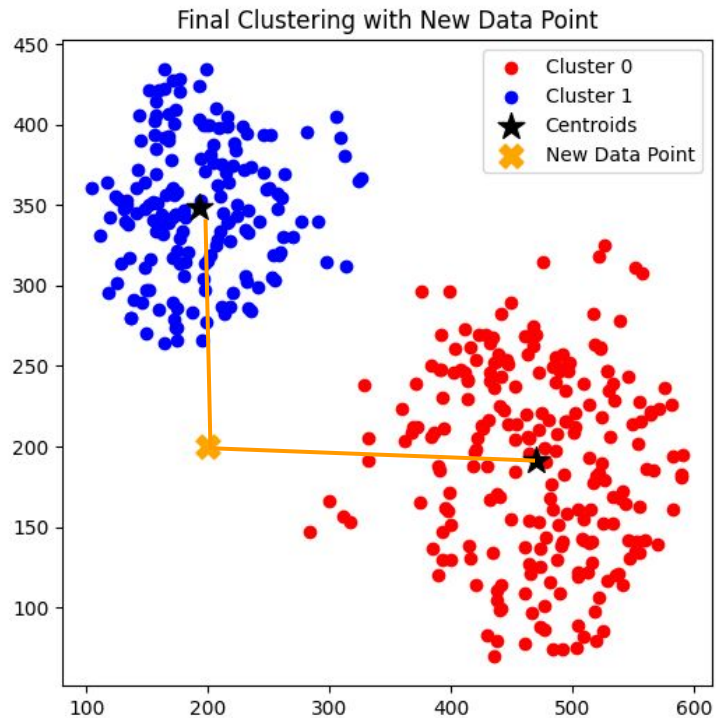
1. Initialization: Randomly select k centroids.
2. Assignment: Assign each data point to the nearest centroid.
3. Update: Recompute centroids as the mean of the points assigned.
4. Iterate: Repeat until convergence (no change in assignments).

K-means clustering

What happens if we get a new data point in the K-means algorithm after training?

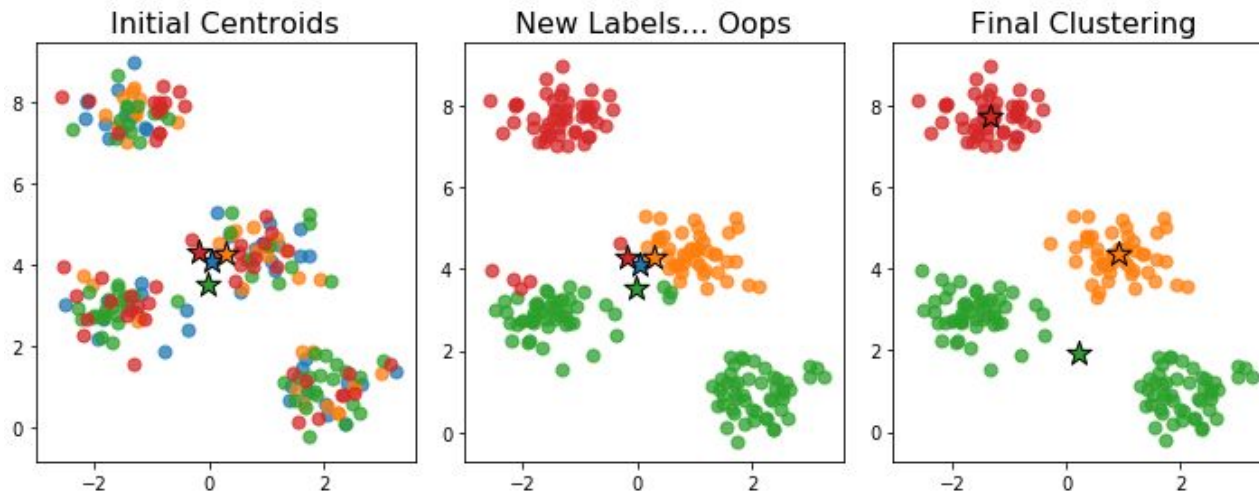
The k-means algorithm would have saved the coordinates of the centroid. It clusters new data points by computing the euclidean distance between the new data points and all the centroids. Then, it chooses the cluster whose centroid is closest to the new data point.

K-means clustering



K-means clustering

The dataset we saw in the example was simple. As data distributions get more complex, we might not converge to the best clusters. **K-means clustering is sensitive to the random initialization of the centroids.**



K-means clustering

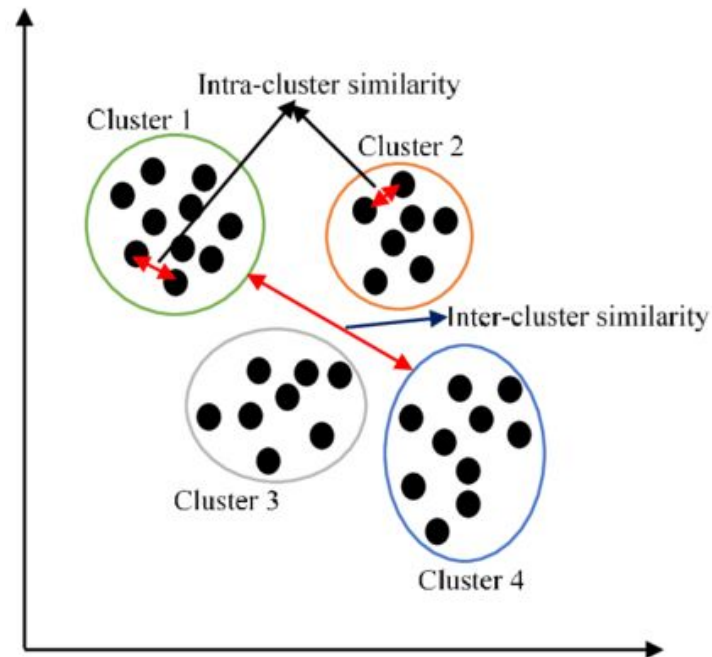
We would then have two issues:

1. What is the appropriate k for my dataset?
2. How do I ensure that my clusters are the most informative/optimal clusters?

Clustering Evaluation Metrics

To validate the quality of our clusters, we can use different metrics such as:

1. **Intra-cluster cohesion** (compactness):
Measures how near the data points in a cluster are to the cluster centroid.
2. **Inter-cluster separation** (isolation):
Measures how much clusters are far from each other.



K-means Lab ‘k-means clustering’

DBSCAN algorithm

What is **DBSCAN**?

- A density-based clustering algorithm that finds clusters of arbitrary shapes.
- Capable of identifying noise (outliers) in the dataset.

What do we mean by **density-based**?

- Clusters are regions of high density **separated** by regions of low density.

DBSCAN algorithm



Advantages of DBSCAN

Flexible Cluster Shapes

DBSCAN can identify clusters of various shapes



No Need to Specify Number of Clusters

DBSCAN does not require pre-defined clusters like K-means



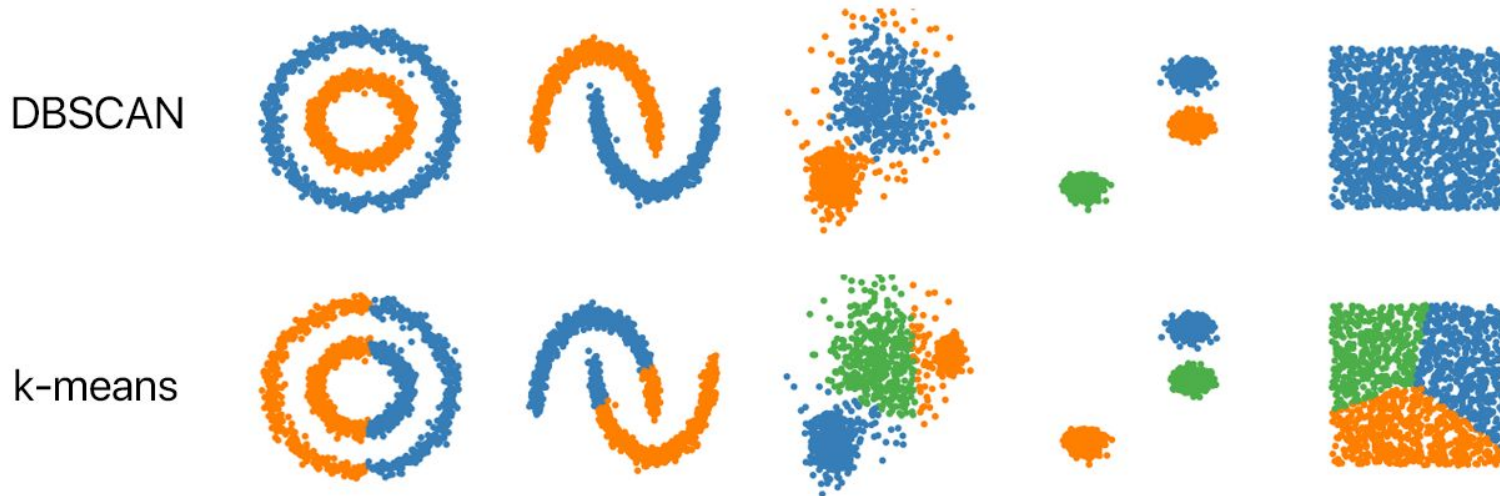
Ability to Identify Noise

DBSCAN effectively detects outliers in datasets



DBSCAN algorithm

DBSCAN algorithm can overcome some of the shortcomings of k-means clustering.



DBSCAN algorithm

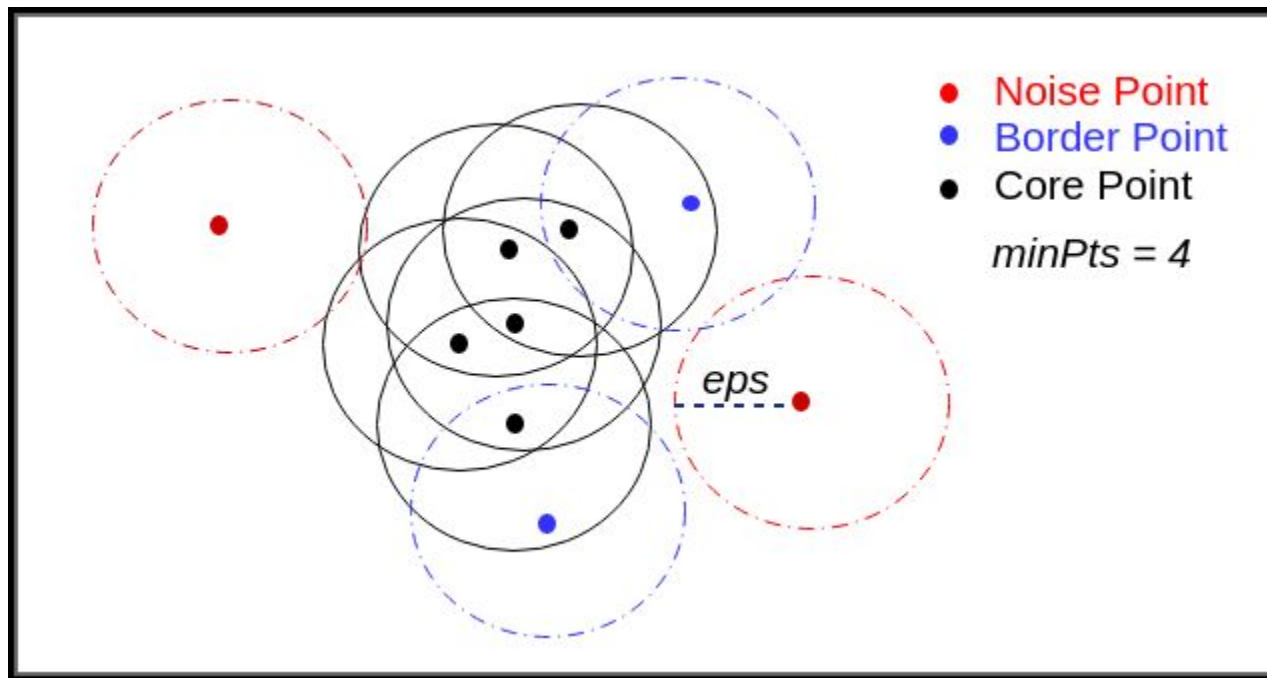
DBSCAN has two main parameters that need to be defined:

1. **Epsilon (ϵ)**: The neighborhood radius used to search for nearby points.
2. **MinPts**: The minimum number of points required to form a dense region.

From these metrics, we can conclude three types of points:

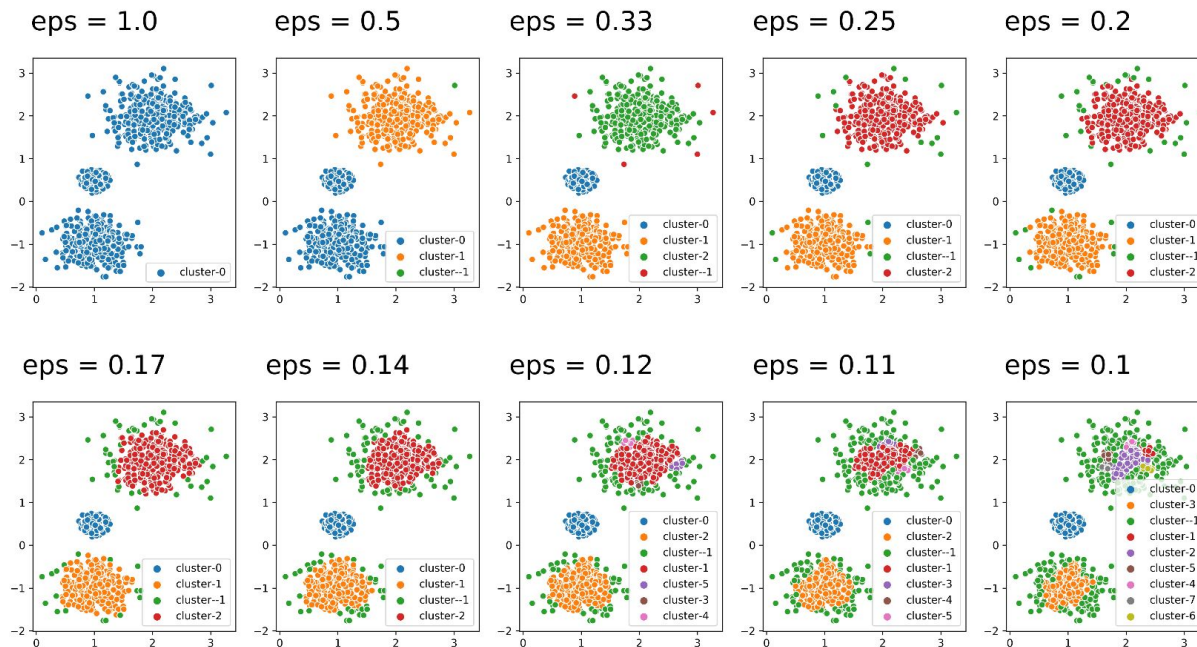
3. **Core Points**: A point is considered a core point if it has at least a minimum number of points (MinPts) within a specified radius (epsilon, ϵ).
4. **Border Points**: A point that is not a core point but falls within the neighborhood of a core point.
5. **Noise Points**: Points that are neither core points nor border points are classified as noise.

DBSCAN algorithm



DBSCAN algorithm

The choice of epsilon affects the outcome of the model:



DBSCAN algorithm

So, how does the algorithm work?

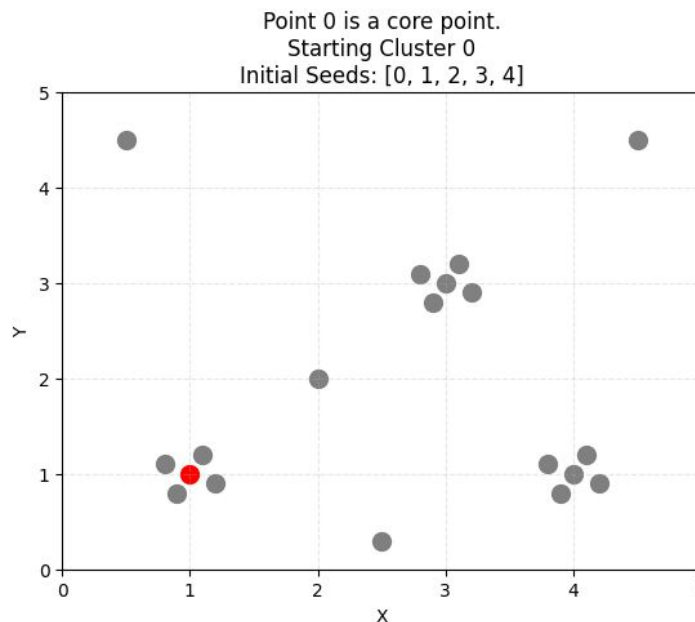
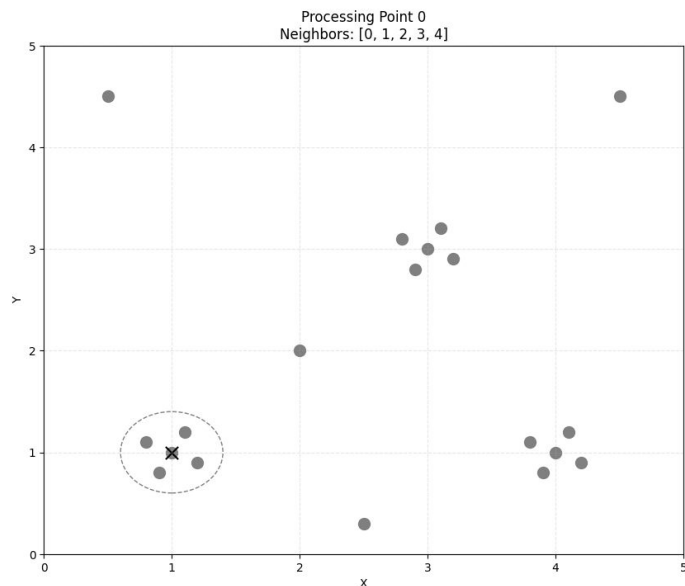
1. For each point in the dataset, determine if it is a core point by checking if it has **at least MinPts within the ϵ radius**.

Let's assume $\epsilon=1$ and MinPts = 3:

- You draw a circle of radius 1 around the selected point.
- Any point that lies within (or on) that circle is part of the neighborhood.
- The selected point itself is always included, so the neighborhood size is at least 1 (just itself).
- If there are **2 or more** other points also within distance 1, then the neighborhood size becomes 3 or more, which meets the core point criterion

DBSCAN algorithm

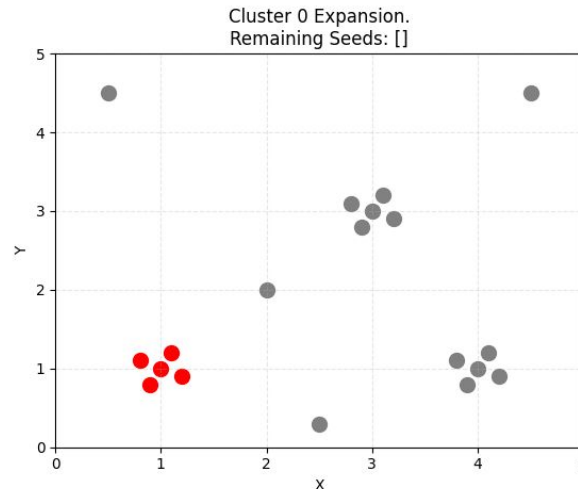
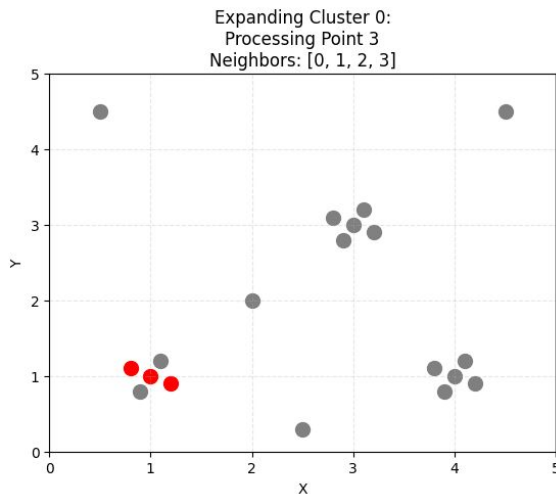
2. If a point is a core point, create a new cluster and include all points within its ϵ neighborhood.





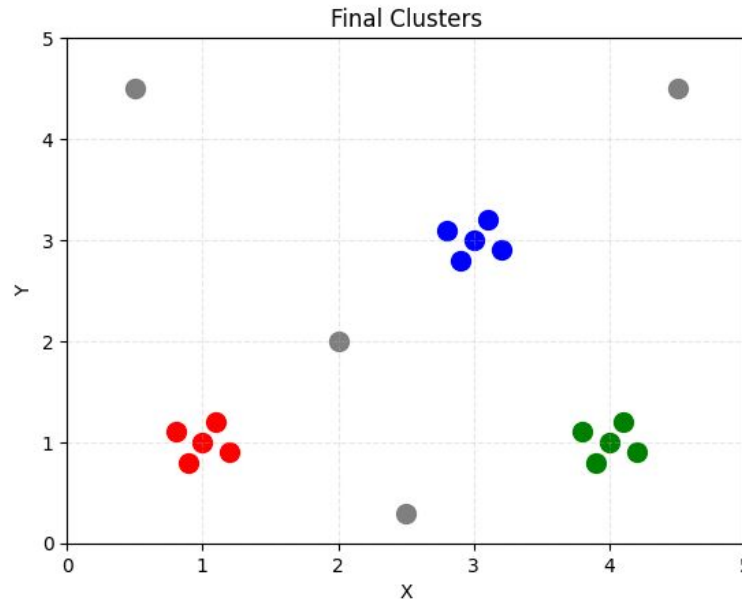
DBSCAN algorithm

3. Expand the cluster by recursively checking the neighbors of each core point in the cluster.



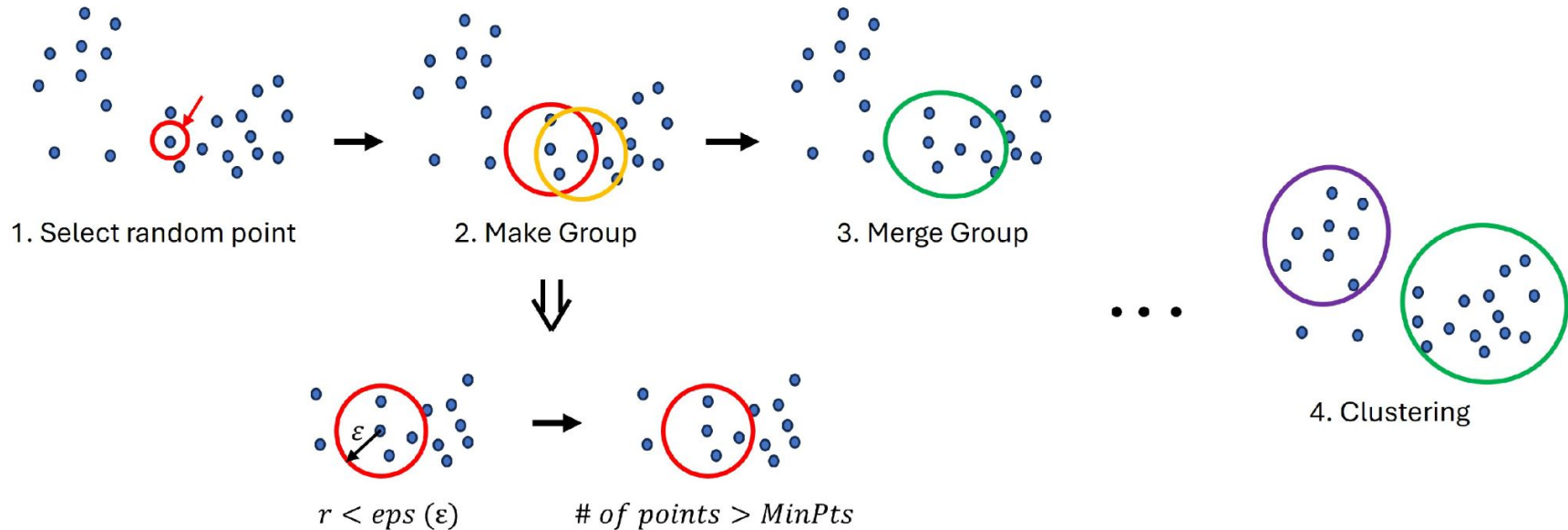
DBSCAN algorithm

4. If a point is a border point, assign it to the cluster of its nearest core point.
5. Repeat the process until all points have been processed.



DBSCAN algorithm

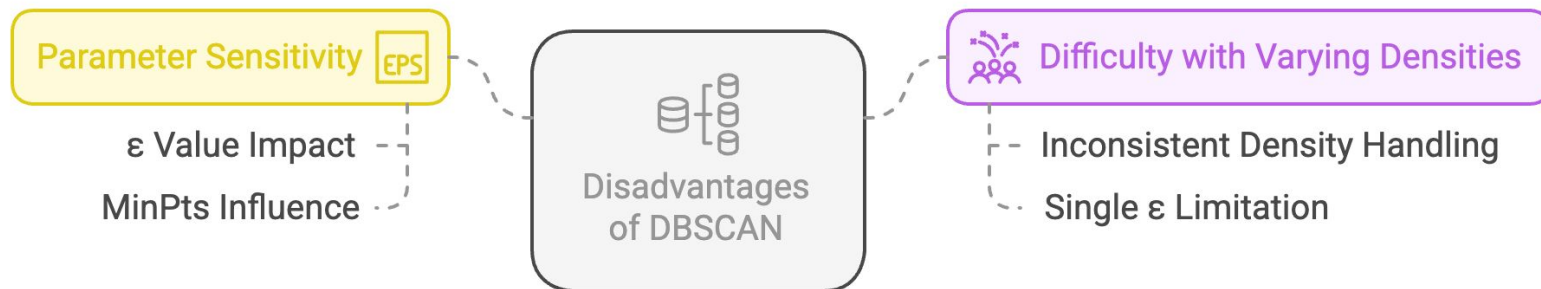
To summarize, the DBSCAN algorithm trains through the following steps:



DBSCAN algorithm



Disadvantages of DBSCAN Clustering Algorithm



DBSCAN Lab 'DBSCAN'

Hierarchical Clustering

A hierarchical clustering approach is based on the **determination of successive clusters based on previously defined clusters**. It's a technique aimed more toward grouping data into a **tree of clusters called dendrograms**, which graphically represents the hierarchical relationship between the underlying clusters.

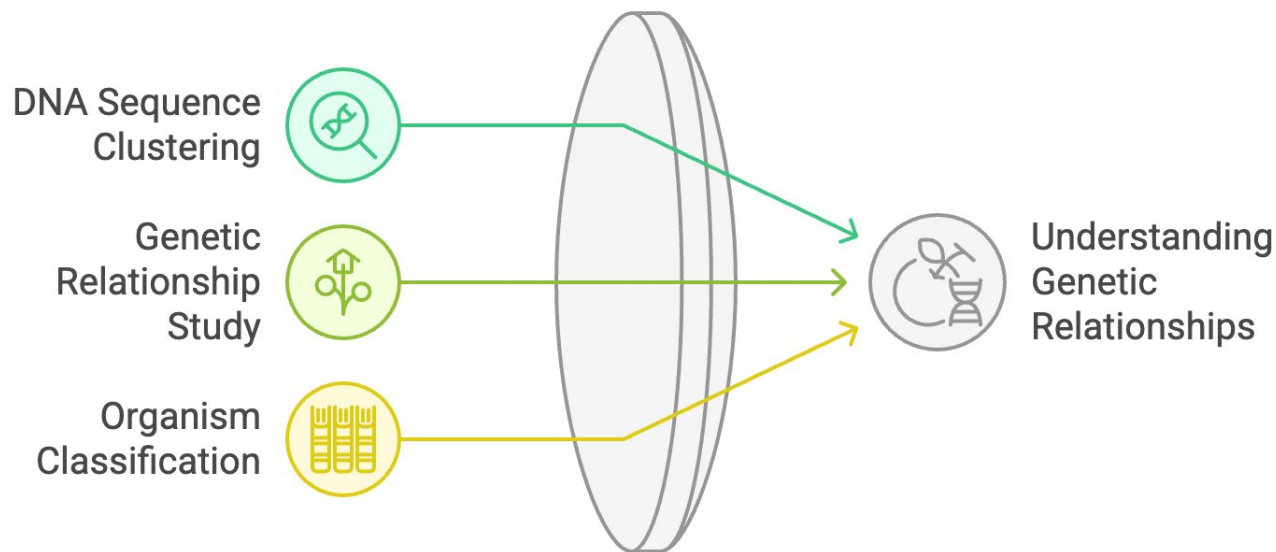
There are two main approaches:

1. Agglomerative (Bottom-Up)
2. Divisive (Top-Down)

Applications of Hierarchical Clustering

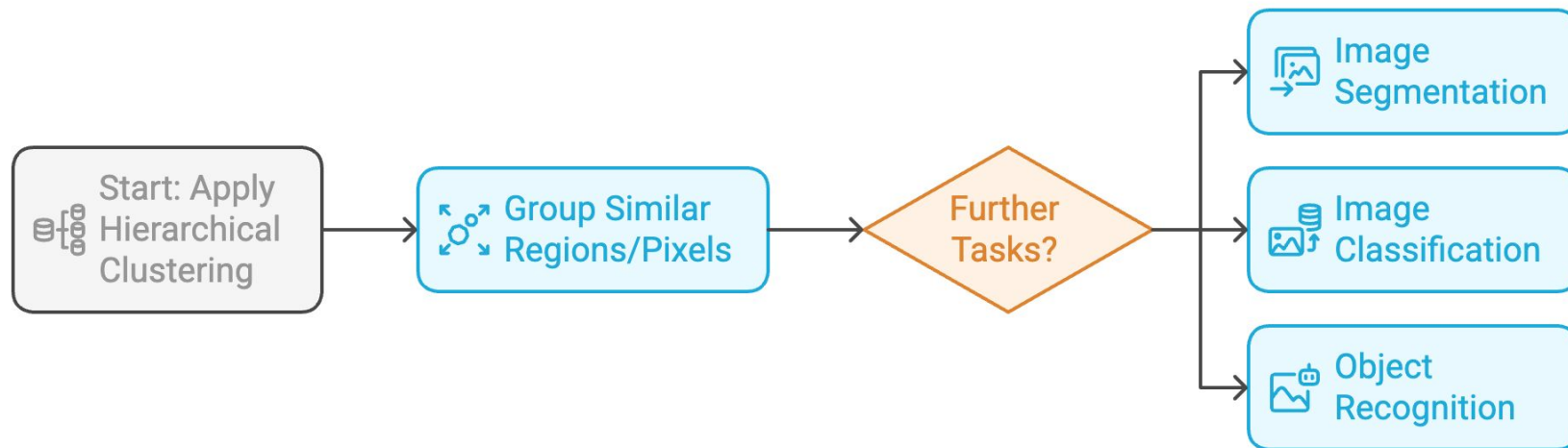


Hierarchical Clustering in Biology



Applications of Hierarchical Clustering

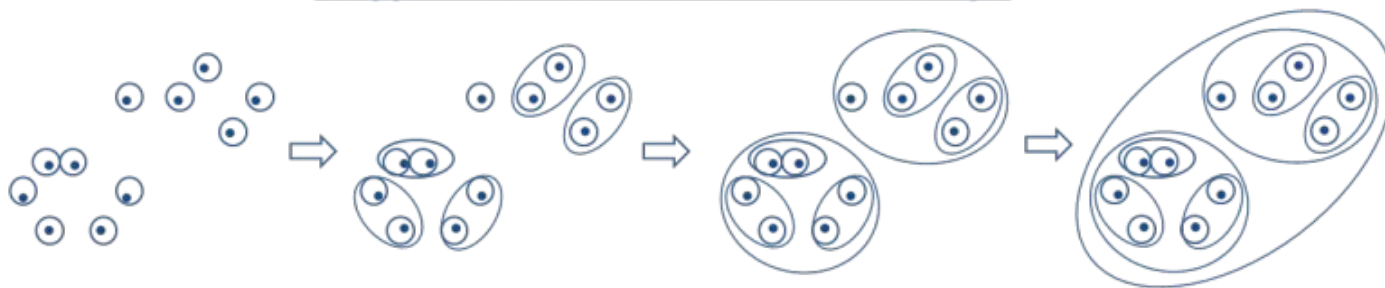
Hierarchical Clustering in Image Processing



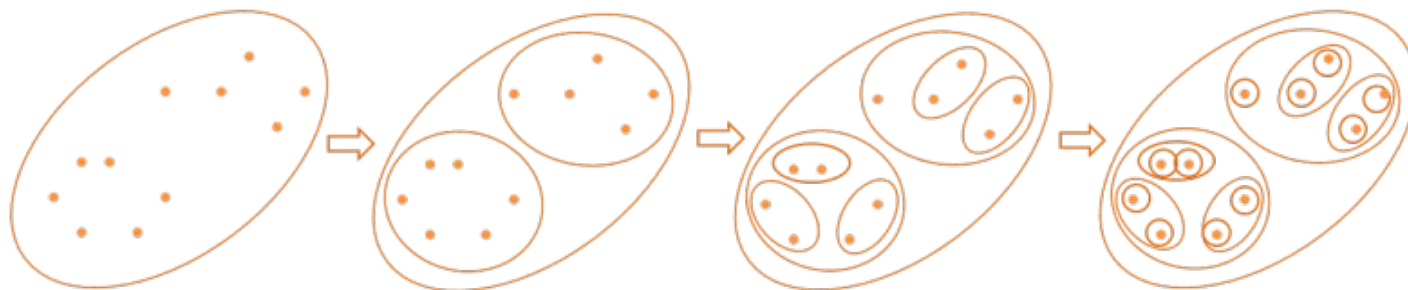
Hierarchical Clustering



Agglomerative Hierarchical Clustering



Divisive Hierarchical Clustering



Hierarchical Clustering - Divisive

Divisive (Top-Down):

- **Start:** All data points begin in one cluster.
- **Process:** At each step, the algorithm splits the cluster into two (or more) sub-clusters.
- **Output:** A dendrogram that shows how the single cluster is split into finer groups.

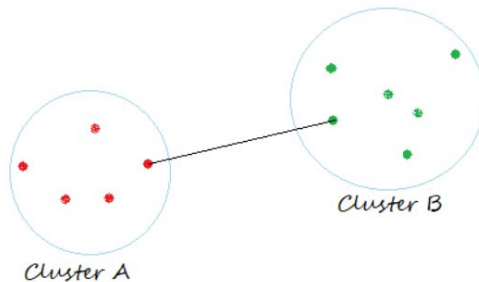
Hierarchical Clustering - Divisive

To perform divisive clustering, we need to define two metrics:

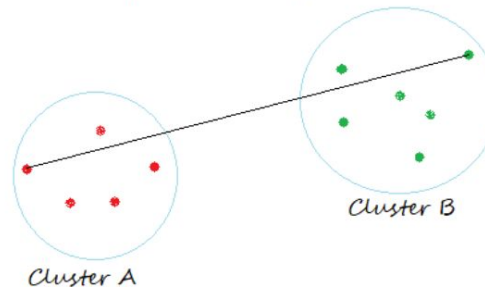
1. **Distance Metric:** Determines how the distance between two **points** (or clusters) is measured. Common metrics include Euclidean, Manhattan, or cosine distance.
2. **Linkage Criteria:** Determines how the distance between **clusters** is defined. Popular choices are:
 - a. **Single Linkage:** Distance between the two closest points (one in each cluster).
 - b. **Complete Linkage:** Distance between the two farthest points.
 - c. **Average Linkage:** Average distance between all pairs of points (one from each cluster).
 - d. **Centroid Linkage:** Distance between cluster centroids

Hierarchical Clustering - Divisive

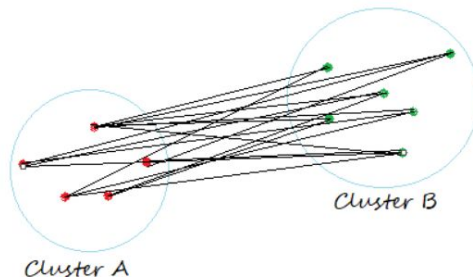
Single Linkage



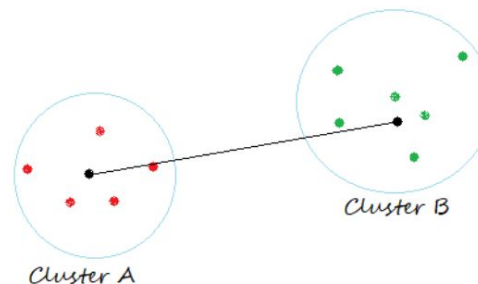
Complete Linkage



Average Linkage



Centroid Linkage



Hierarchical Clustering - Divisive

The steps in executive divisive clustering:

1. Initialization: Begin with all data points in one single cluster.

Suppose we have the following pair-wise distances in our dataset. Initial Cluster: {A, B, C, D}

	A	B	C	D
A	0	1	3	4
B	1	0	2	5
C	3	2	0	3
D	4	5	3	0

Hierarchical Clustering - Divisive

2. Compute Average Dissimilarity for Each Point in the Cluster:

- **A:** $\text{Avg}(d(A, B) = 1, d(A, C) = 3, d(A, D) = 4) = \frac{1+3+4}{3} \approx 2.67$
- **B:** $\text{Avg}(d(B, A) = 1, d(B, C) = 2, d(B, D) = 5) = \frac{1+2+5}{3} \approx 2.67$
- **C:** $\text{Avg}(d(C, A) = 3, d(C, B) = 2, d(C, D) = 3) = \frac{3+2+3}{3} \approx 2.67$
- **D:** $\text{Avg}(d(D, A) = 4, d(D, B) = 5, d(D, C) = 3) = \frac{4+5+3}{3} = 4$

Point D has the highest average dissimilarity (4). Thus, D is the most “outlying” object in the cluster.

Hierarchical Clustering - Divisive

3. Split the Cluster Using the Most Dissimilar Point:

New Clusters:

- Cluster 1: {D} (the splitting object)
- Cluster 2: {A, B, C} (the remaining points)

Hierarchical Clustering - Divisive

4. Consider Further Splitting of the Larger Cluster {A, B, C}:

- A: $A_{vg}(1, 3) = 2$
- B: $A_{vg}(1, 2) = 1.5$
- C: $A_{vg}(3, 2) = 2.5$

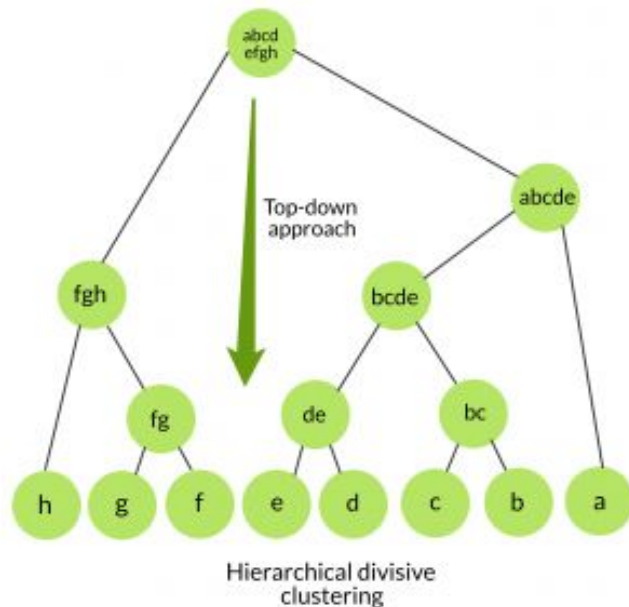
Point C has the highest average dissimilarity (2.5) within {A, B, C}.

Split Cluster {A, B, C}:

- Cluster 2a: {C} (splitting object)
- Cluster 2b: {A, B} (remaining points)

Hierarchical Clustering - Divisive

5. Repeat: Apply the same splitting process recursively to the clusters that still exhibit significant heterogeneity. Stop when a stopping criterion is reached (e.g., each cluster has only one element, or the within-cluster dissimilarity falls below a threshold).



Hierarchical Clustering

Agglomerative (Bottom-Up):

- **Start:** Each data point begins as its own cluster.
- **Process:** At each step, the two most similar (or closest) clusters are merged into one. This process is repeated until all points are merged into a single cluster (or until a stopping criterion is met).
- **Output:** A dendrogram (tree-like diagram) that shows how clusters are merged over the iterations.

Choosing your clustering algorithm

Algorithm	Key Parameters	Pros	Cons	Best For
K-means	k, initialization	Simple, scalable, fast	Requires k, assumes spherical clusters	Clearly separable, convex clusters
DBSCAN	ϵ , MinPts	Discovers arbitrary shapes, robust to noise	Parameter sensitivity, struggles with varying density	Datasets with noise and irregular shapes
Hierarchical	Linkage method	No need for k, produces dendrogram	Computationally heavy, not scalable	Exploratory analysis, small to medium datasets

Dimensionality Reduction

Dimensionality Reduction

As we mentioned before, datasets can get quite huge and complex. This characteristic makes it hard to process, model, and visualize this type of data.

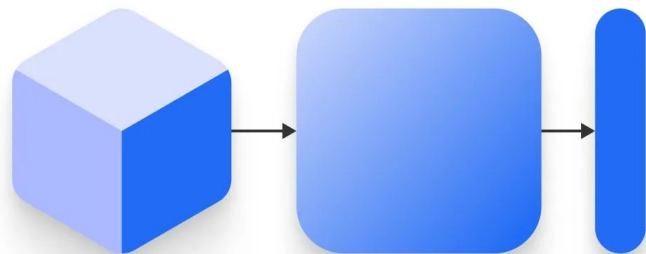


Dimensionality Reduction

Dimensionality reduction is the process of reducing the number of features (or dimensions) in a dataset while retaining as much information as possible.

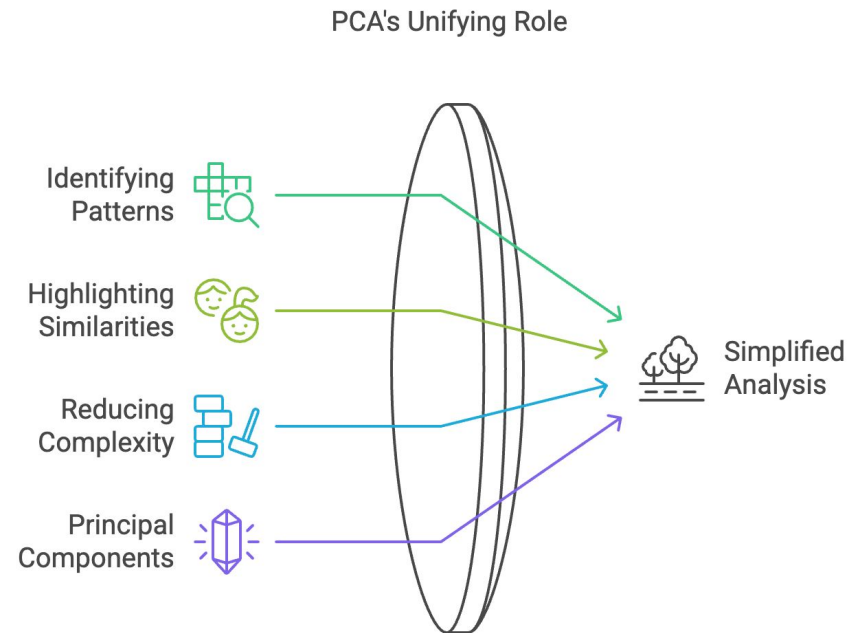
How do we reduce the number of features while retaining the information? This is where unsupervised dimensionality reduction algorithms come in.

Dimensionality Reduction



Principal Component Analysis

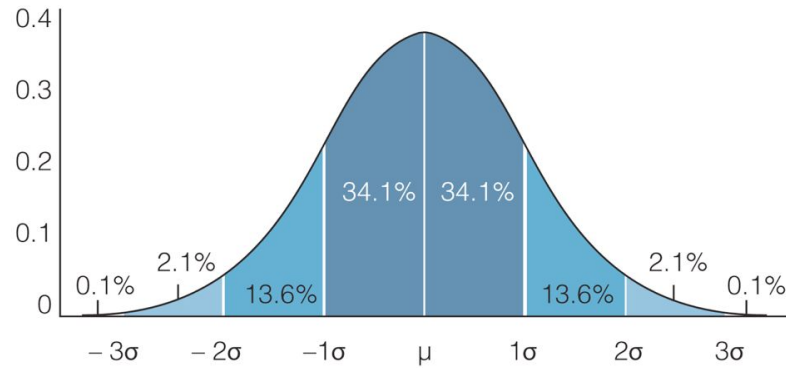
Principal Component Analysis (PCA) is a powerful statistical technique used for dimensionality reduction while preserving as much variance as possible in the data. It transforms a large set of variables into a smaller one that still contains most of the information in the original dataset.



Principal Component Analysis

The core idea of PCA is to find the directions (principal components) along which the variance of the data is maximized. Variance is the spread between numbers in a data set. It determines how far each number is from the mean and from every other number in the set.

Distribution of Variance



μ = Expected Value

- 1 σ to 1 σ = 1 Standard Deviation (ie: ~2/3 of the time, your results/variance will fall within this range)
- 2 σ to 2 σ = 2 Standard Deviations (ie: 95% of the time, your results/variance will fall within this range)
- 3 σ to 3 σ = 3 Standard Deviations (ie: 99.7 of the time, your results/variance will fall within this range)

Principal Component Analysis

The steps involved in PCA include:

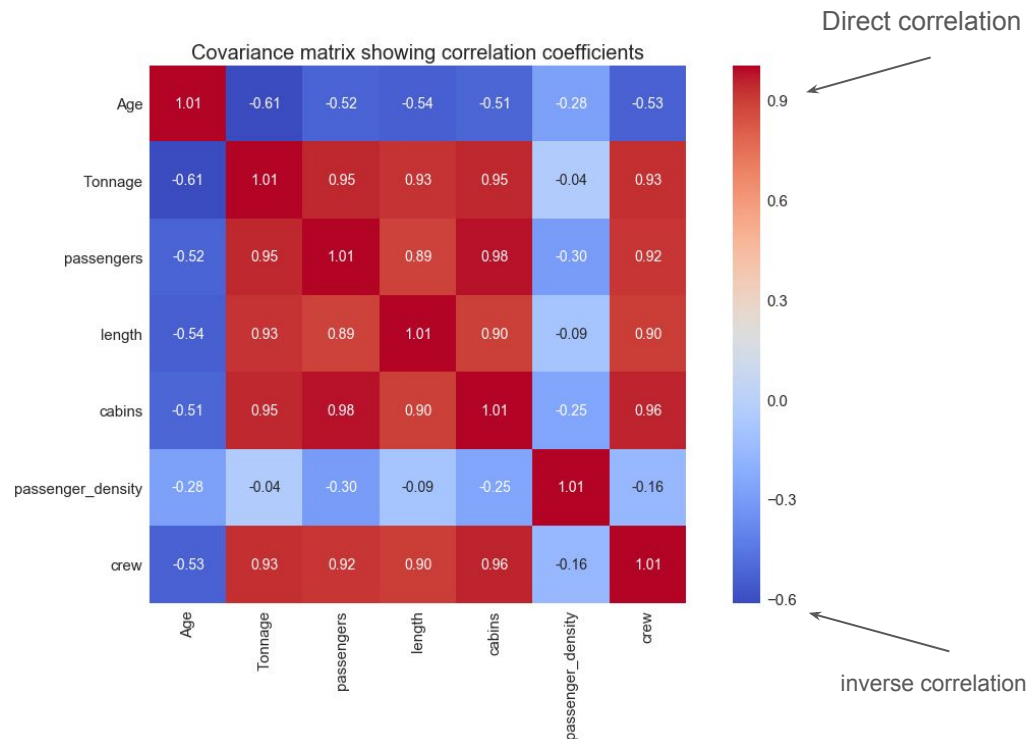
1. Standardization: Center the data by subtracting the mean and scaling to unit variance.

$$z = \frac{value - mean}{standard\ deviation}$$

Principal Component Analysis

2. Covariance Matrix Computation:
Calculate the covariance matrix to understand how the variables vary together.

The covariance matrix, also known as the correlation matrix, shows the correlation between the features of the dataset. This helps us identify redundant information immediately.



Principal Component Analysis

New Directions (Eigenvectors)

- **What it means:** Think of your data as a cloud of points. PCA finds new axes (directions) where this cloud is stretched out the most.
- **In simple terms:** Eigenvectors are these new axes or directions that capture the trends in the data.

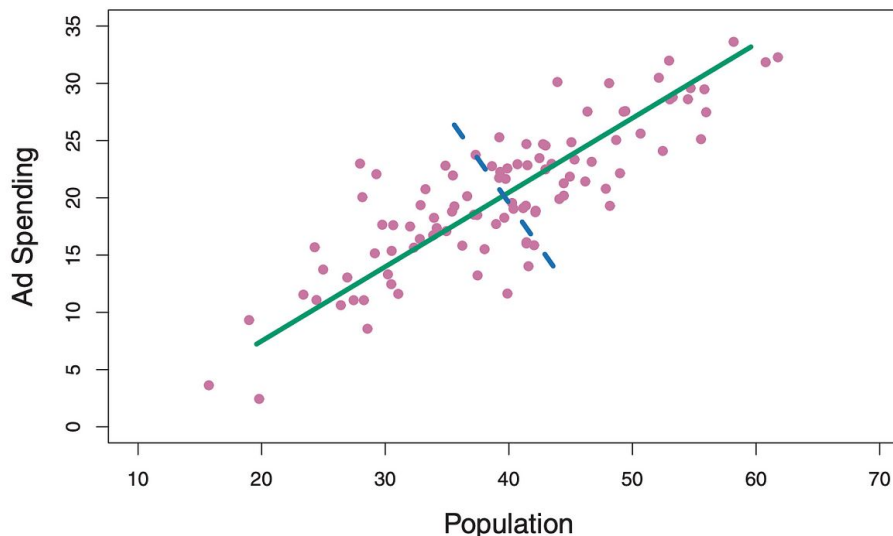
Principal Component Analysis

How Much is Important (Eigenvalues)

- **What it means:** Each eigenvalue tells you how “long” or “spread out” the data is along its corresponding eigenvector.
- **In simple terms:** A larger eigenvalue means that the direction (eigenvector) captures more variation in the data, making it more important for summarizing the data structure.

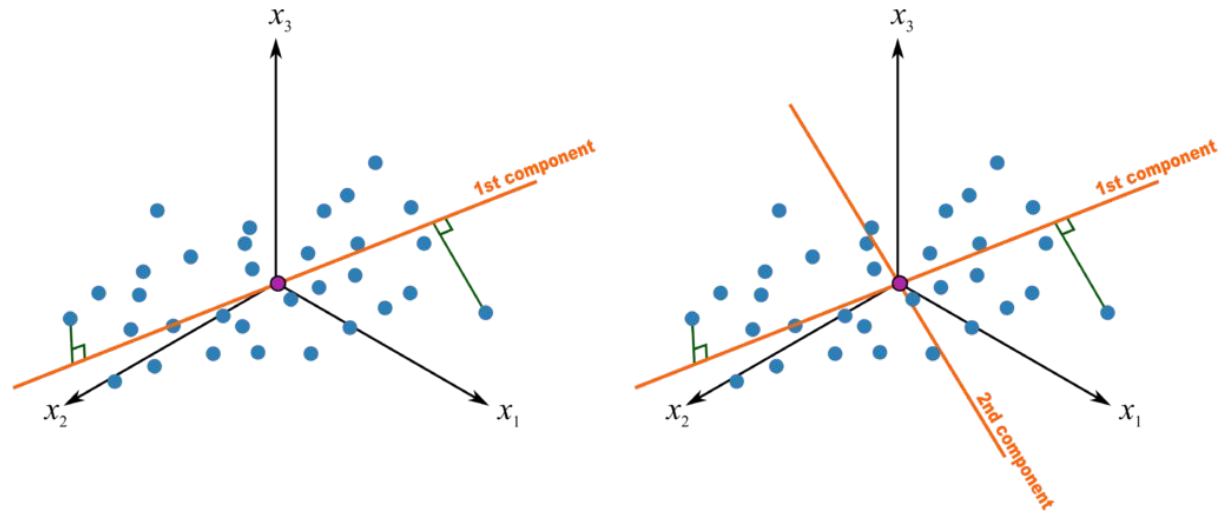
Principal Component Analysis

3. Eigenvalue and Eigenvector Calculation: Determine the eigenvalues and eigenvectors of the covariance matrix. The eigenvectors represent the directions of the principal components, while the eigenvalues indicate the amount of variance captured by each component.



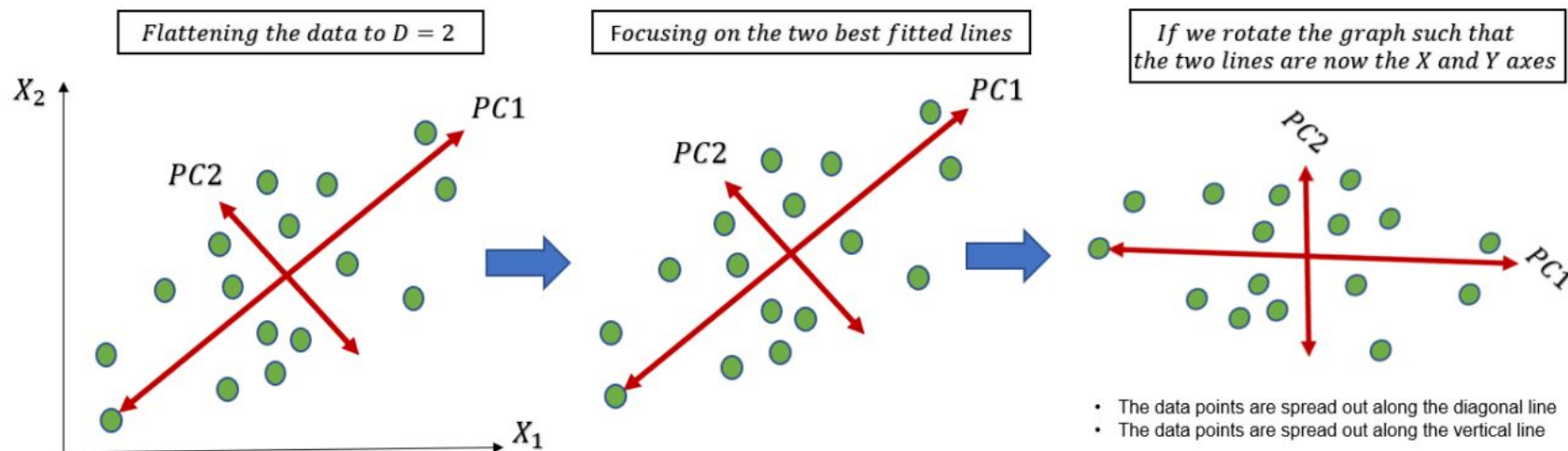
Principal Component Analysis

4. Create Feature Vector: Computing the eigenvectors and ordering them by their eigenvalues in descending order, allow us to find the principal components in order of significance. In this step, what we do is, to choose whether to keep all these components or discard those of lesser significance (of low eigenvalues), and form with the remaining ones a matrix of vectors that we call Feature vector.



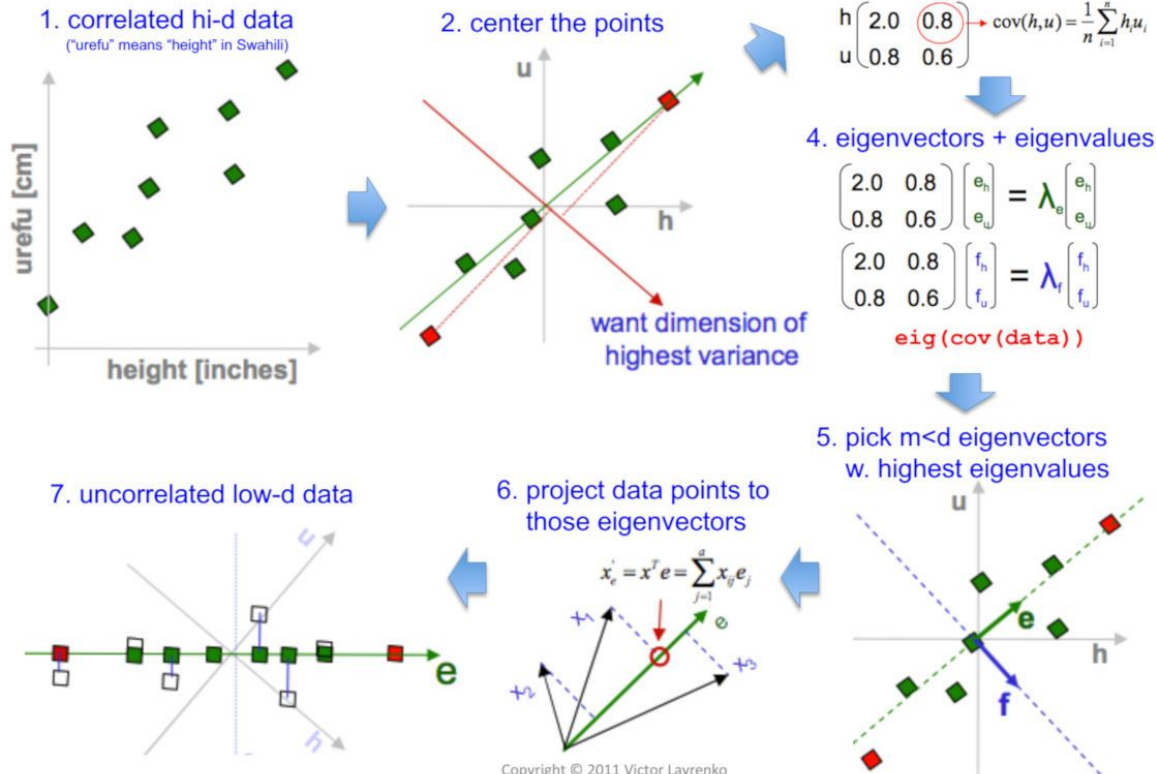
Principal Component Analysis

5. Transform the data to a lower dimension: Project the original data onto the new feature space using the selected principal components.



Principal Component Analysis Summary

PCA in a nutshell



PCA Lab 'PCA'



Thank you